

EcoStay

A Smart Sustainable Accommodation Recommendation System

FULL IMPLEMENTATION PLAN

Project Type: Full-stack responsive web application / PWA

Recommended Stack: React + TypeScript + Tailwind CSS + Node.js/Express + MySQL

Core Theme: Sustainable travel, accommodation discovery, personalization and transparent sustainability scoring

1. Executive Overview

EcoStay is a full-stack web application that helps users discover and compare accommodations using both conventional travel criteria and sustainability criteria. Instead of ranking properties only by price, location and rating, the system exposes sustainability attributes such as renewable energy, water conservation, waste management, energy efficiency, green construction and local-community support.

The recommended implementation is a responsive web application that works on desktop, tablet and mobile. A Progressive Web App (PWA) layer can later provide an app-like experience without requiring a separate native Android/iOS codebase.

Primary objective

Build a working platform where a user can enter a destination, budget and sustainability preferences; receive explainable recommendations; inspect an accommodation's sustainability profile; compare properties; save favorites; and manage bookings or contact requests.

Project outcomes

- Modern UI/UX prototype and responsive production interface.
- Secure authentication and role-based access.
- Normalized MySQL database for users, accommodations, sustainability data, reviews and bookings.
- REST API connecting the frontend and backend.
- Explainable weighted recommendation engine.
- Admin dashboard for accommodation and sustainability-data management.
- Testing, deployment, documentation and project demonstration.

2. Problem Statement and Scope

Travelers often have difficulty identifying genuinely sustainable accommodation because sustainability information is fragmented, inconsistent or difficult to compare. Conventional booking interfaces emphasize price, location and reviews, while environmental and community-impact factors are often secondary.

In scope

- User registration, login and profile.
- Destination, date, guest and budget search.
- Sustainability preference selection.
- Accommodation listing and detailed profiles.
- Sustainability score and category breakdown.
- Personalized recommendations.
- Compare, wishlist and reviews.
- Booking/contact workflow (demo or API-ready).
- Admin dashboard and CRUD operations.
- Responsive design and PWA-ready architecture.

Out of scope for the first release

- Direct payment processing.
- Guaranteed real-time hotel inventory unless a third-party booking API is integrated.

- Independent certification of sustainability claims.
- Complex machine-learning personalization requiring a large historical dataset.

3. Methodology

Recommended development methodology: Agile + iterative prototyping

Use short development iterations. Each iteration should produce a demonstrable feature set. Begin with UI/UX and database design, then implement authentication, accommodation management, search, recommendation logic, user features, admin features and finally testing/deployment.

Sprint	Main deliverable	Acceptance criteria
1	Requirements + UX	Problem, personas, user journeys and wireframes approved
2	Frontend foundation	Responsive layout, navigation and design system working
3	Backend + database	API, MySQL schema and authentication connected
4	Accommodation module	Search, filters, details and admin CRUD working
5	Recommendation engine	Scores and explainable recommendations generated
6	User features	Wishlist, compare, reviews and booking/contact flow
7	Admin + analytics	Dashboard, moderation and sustainability data management
8	Testing + deployment	Responsive, secure, tested and deployed build

4. UI/UX Design Plan

Design direction

Use a clean eco-tech visual language: natural greens for sustainability actions, neutral backgrounds for readability, blue for water-related indicators, warm accent tones for energy/community categories, rounded cards, clear iconography and strong accessibility contrast.

Token	Suggested value	Use
Primary	#2E7D32	Buttons, active states, sustainability score
Dark Green	#1B5E20	Headings and brand
Light Green	#E8F5E9	Cards, badges and backgrounds
Background	#F8FAF7	Main application background
Text	#263238	Primary text
White	#FFFFFF	Cards and surfaces

Core screens

- Landing/Home: value proposition, search, featured eco-stays and how-it-works.
- Search & Preferences: destination, dates, budget, accommodation type and sustainability priorities.
- Recommendation Results: cards, EcoScore, matched preferences, filters, sort and compare selection.
- Accommodation Details: gallery, price, rating, sustainability breakdown, amenities, reviews and booking/contact action.
- Compare: side-by-side sustainability, price, rating and amenities.

- Wishlist/Profile: saved properties, recent searches, preferences and bookings.
- Admin Dashboard: KPIs, property management, sustainability verification, users and reviews.

UX principles

- Progressive disclosure: show the most important information first.
- Explainability: every recommendation should show why it was suggested.
- Consistency: reusable cards, buttons, badges, forms and navigation.
- Accessibility: keyboard navigation, readable text, semantic HTML, alt text and adequate contrast.
- Mobile-first responsive layout with bottom navigation on small screens.
- Avoid misleading sustainability claims; show the source/status of sustainability information where possible.

5. UI Structure / Information Architecture

```
EcoStay
├── Public
│   ├── Home
│   ├── Explore
│   ├── How It Works
│   ├── Accommodation Details
│   ├── Login / Register
│   └── User
│       ├── Dashboard
│       ├── Recommendations
│       ├── Compare
│       ├── Wishlist
│       ├── Bookings
│       └── Profile / Preferences
├── Admin
│   ├── Dashboard
│   ├── Accommodations
│   ├── Sustainability Data
│   ├── Users
│   ├── Reviews
│   └── Reports
├── System
├── Authentication
├── Recommendation Engine
├── Sustainability Scoring
└── REST API
```

6. Frontend Implementation

Technology

Technology	Purpose
React + TypeScript	Component-based UI and type-safe application logic
Vite	Fast development/build tooling
Tailwind CSS	Responsive styling and design tokens
React Router	Client-side routing
Lucide React	Consistent interface icons
TanStack Query	API fetching, caching and server-state management
React Hook Form + Zod	Form handling and validation
Recharts	Admin analytics and sustainability charts

Suggested component structure

```
src/
├── components/
│   ├── ui/
│   ├── accommodation/
│   ├── sustainability/
│   ├── search/
│   ├── layout/
│   ├── pages/
│   ├── features/
│   ├── auth/
│   ├── recommendations/
│   ├── wishlist/
│   ├── compare/
│   ├── bookings/
│   ├── services/
│   ├── hooks/
│   ├── types/
│   ├── utils/
│   └── routes/
```

Frontend workflow

UI event → validate input → call API → show loading state → receive response → update cached state → render success/error state. Use skeleton loaders and empty states so the application remains understandable while data is loading.

7. Backend Architecture

Technology

Layer	Implementation
Runtime	Node.js
Framework	Express.js
Database	MySQL
Authentication	JWT access tokens + secure password hashing
Validation	Zod/Joi or equivalent server-side schema validation
API style	RESTful JSON APIs
Security	Helmet, CORS policy, rate limiting, parameterized SQL, input validation
Logging	Structured request/error logging

Backend modules

- Auth Service: registration, login, refresh/logout strategy and role checks.
- Accommodation Service: properties, location, pricing, amenities and images.
- Sustainability Service: sustainability attributes, evidence/status and score calculation.
- Recommendation Service: preference matching and ranking.
- Review Service: ratings, comments and moderation.
- Wishlist Service: save/remove accommodation.
- Booking Service: booking/contact records and status.
- Admin Service: CRUD, verification and dashboard metrics.

8. Database Design

Table	Important fields
users	id, name, email, password_hash, role, created_at
accommodations	id, name, location, description, price_per_night, rating, status
sustainability	id, accommodation_id, solar, water_conservation, waste_management, rainwater, energy_efficiency, local_support
amenities	id, name
accommodation_amenities	accommodation_id, amenity_id
reviews	id, user_id, accommodation_id, rating, comment, status, created_at
wishlists	id, user_id, accommodation_id, created_at
bookings	id, user_id, accommodation_id, check_in, check_out, guests, status
images	id, accommodation_id, image_url, sort_order
search_history	id, user_id, destination, filters_json, created_at

Relationships

One user can have many reviews, wishlist items, bookings and searches. One accommodation can have many reviews, images and amenities. Sustainability data is associated with an accommodation. The many-to-many accommodation/amenity relationship uses a junction table.

9. Recommendation Engine

Start with an explainable weighted scoring model rather than machine learning. This is easier to validate with a small dataset and easier to defend in a project presentation.

Example scoring model

```
Final Recommendation Score =  
0.40 × Sustainability Score  
+ 0.25 × Preference Match  
+ 0.15 × Location Match  
+ 0.10 × Budget Match  
+ 0.10 × User Rating
```

Sustainability score

```
Sustainability Score =  
25% Renewable/Energy  
20% Water Conservation  
20% Waste Management  
15% Energy Efficiency  
10% Local Community Support  
10% Green Construction
```

Recommendation flow

- Collect user criteria.
- Apply hard filters such as destination and maximum budget.
- Normalize each remaining property into comparable 0–100 values.
- Calculate preference-match score.
- Calculate sustainability score.
- Combine weighted scores.

- Sort by final score.
- Return top results plus explanation tags such as “Matches your solar preference”.

Important: sustainability information should be presented as reported/verified data, not as an independently certified environmental claim unless an actual certification source is available.

10. REST API Plan

Method	Endpoint	Purpose
POST	/api/auth/register	Create user
POST	/api/auth/login	Authenticate user
GET	/api/accommodations	Search/filter accommodations
GET	/api/accommodations/:id	Get property details
POST	/api/recommendations	Generate recommendations
GET	/api/wishlist	Get saved stays
POST	/api/wishlist/:id	Save a stay
DELETE	/api/wishlist/:id	Remove saved stay
POST	/api/reviews	Create review
GET	/api/compare?ids=...	Compare properties
POST	/api/bookings	Create booking/contact request
GET	/api/admin/dashboard	Admin metrics
POST/PUT/DELETE	/api/admin/accommodations	Admin CRUD

11. Security and Privacy

- Hash passwords using a modern password-hashing algorithm; never store plain-text passwords.
- Validate all request bodies, query parameters and route parameters on the server.
- Use parameterized queries/ORM methods to prevent SQL injection.
- Apply role-based authorization to admin endpoints.
- Configure CORS for known frontend origins.
- Use HTTPS in production and secure cookie/token practices.
- Rate-limit login and other abuse-sensitive endpoints.
- Do not expose internal error details or database credentials to clients.
- Minimize stored personal data and provide appropriate account/data-management controls.

12. Testing Strategy

Testing level	Examples
Unit testing	Score calculation, validators, utility functions
API testing	Auth, accommodation, recommendations, wishlist and admin endpoints
Integration testing	Frontend-to-API-to-MySQL flows
UI testing	Forms, navigation, filters, comparison and responsive states

Testing level	Examples
Usability testing	Can a first-time user find a sustainable stay without assistance?
Security testing	Authorization, invalid input, rate limiting and injection checks
Performance testing	Search response time, image loading and large result sets

13. Deployment Architecture

```

User Browser / Mobile PWA
↓ HTTPS
Frontend Hosting (React build)
↓ REST API
Backend Hosting (Node.js + Express)
↓ Secure DB connection
MySQL Database
↓
Object/Image Storage

```

A practical deployment can use a managed frontend host, a managed Node.js service and a managed MySQL database. Keep environment variables for database credentials, JWT secrets and API keys outside the source repository.

14. Suggested 8-Week Development Timeline

Week	Work
1	Requirements, personas, user stories, sitemap, wireframes and database ER model
2	Design system, responsive layout, landing page, authentication screens
3	Node/Express setup, MySQL schema, authentication and API foundation
4	Accommodation CRUD, search, filters, details and image handling
5	Sustainability scoring and recommendation engine
6	Wishlist, compare, reviews and booking/contact workflow
7	Admin dashboard, analytics, moderation and PWA responsiveness
8	Testing, bug fixing, deployment, documentation, presentation and demo data

15. Final Deliverables

- Responsive EcoStay web application.
- UI/UX design system and wireframes.
- React + TypeScript frontend source code.
- Node.js + Express backend source code.
- MySQL database schema and seed data.
- Recommendation engine and sustainability scoring logic.
- Admin dashboard.
- API documentation.
- Test cases and results.
- Deployment configuration.

- Project report, presentation and demonstration dataset.

16. Recommended Project Demonstration Flow

For the final presentation, demonstrate one complete user journey instead of showing isolated screens.

1. Open EcoStay landing page
2. Search for a destination
3. Set budget and sustainability preferences
4. Generate recommendations
5. Open a property and inspect its EcoScore
6. Compare two properties
7. Save one to Wishlist
8. Submit a review or demo booking/contact request
9. Sign in as Admin
10. Add/update an accommodation and sustainability data
11. Return to recommendations and show the updated result

17. Future Enhancements

- AI conversational travel assistant.
- Real-time availability through partner APIs.
- Map-based discovery.
- Carbon-footprint estimation per stay.
- Verified sustainability certification integrations.
- Personalized learning from user interactions.
- Multi-language support.
- Native mobile application if the PWA proves insufficient.
- Analytics for accommodation owners and sustainability trends.

18. Conclusion

EcoStay can be implemented as a complete academic full-stack project with a clear problem, measurable recommendation logic, a normalized database, responsive UI/UX, REST APIs, admin management and a practical deployment path. The recommended approach is to launch the first version as a responsive web application/PWA and keep the backend API-oriented so a future mobile application can reuse the same services.

Suggested publication title: “EcoStay: A Smart Sustainable Accommodation Recommendation System”

Suggested tagline: “Stay Better. Travel Greener.”